



哈爾濱工業大學

HARBIN INSTITUTE OF TECHNOLOGY



分布式机器学习(I)

机器学习的计算模式

王宏志

哈尔滨工业大学

计算学部海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

- 1 分布式机器学习概述
- 2 SGD与mini-SGD
- 3 SGD的变体
- 4 目标函数的性质
- 5 梯度下降的收敛性分析

目录

Contents

- 1 分布式机器学习概述
- 2 SGD与mini-SGD
- 3 SGD的变体
- 4 目标函数的性质
- 5 梯度下降的收敛性分析

过去：机器学习简史

- 机器学习并非新事物，其起源可追溯至20世纪50年代
- 1950年：艾伦·图灵提出“思考机器”构想
- 1957年：罗森布拉特提出感知机——神经网络的前身
- 1986年：辛顿提出反向传播算法，实现高效神经网络训练
- 1998年：LeCun团队创建MNIST数据集，并利用SVM实现<1%分类错误率

此后因计算能力不足导致AI进入寒冬期

当下：主流成功

机器学习直到21世纪初才因算力提升和数据丰富成为主流

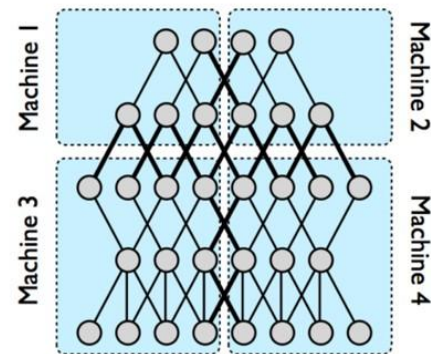
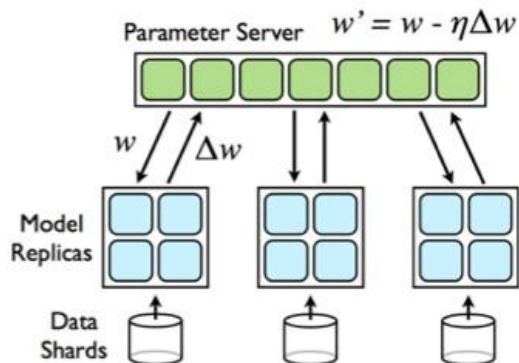
- 2000年代初：亚马逊推出AWS，实现灵活、可扩展且经济的云计算
- 2012年：李飞飞团队创建ImageNet大数据集并举办ILSVRC挑战赛
- 2012年：Krizhevsky团队提出基于卷积神经网络的AlexNet模型，实现ImageNet分类突破
- 2017年：《注意力机制就是你所需的一切》论文由谷歌发表，首次提出Transformer模型架构
- 2022年：OpenAI推出ChatGPT，随后Anthropic的Claude、谷歌的Gemini、Perplexity等公司相继发布生成式AI应用

在这些突破之后，机器学习（ML）迅速发展，并在众多领域取得广泛成功

当前阶段：分布式机器学习

为处理海量训练数据集与模型，必须将计算任务分配至多个节点

- **数据并行分布式ML**：训练集被随机分配到多个节点（云端服务器通常配备GPU），参数服务器聚合来自各节点的梯度
- **模型并行分布式ML**：大型神经网络模型被拆分到不同节点，模型参数在节点间同步

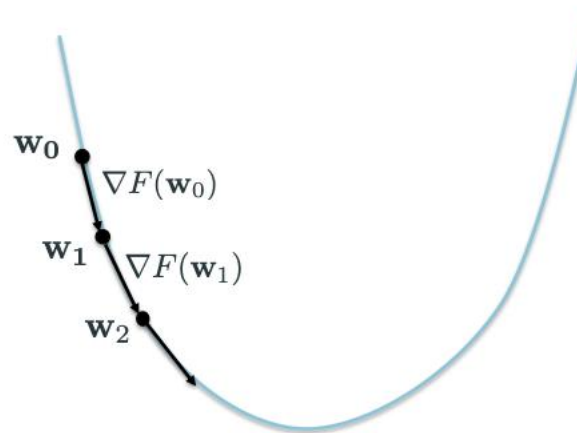
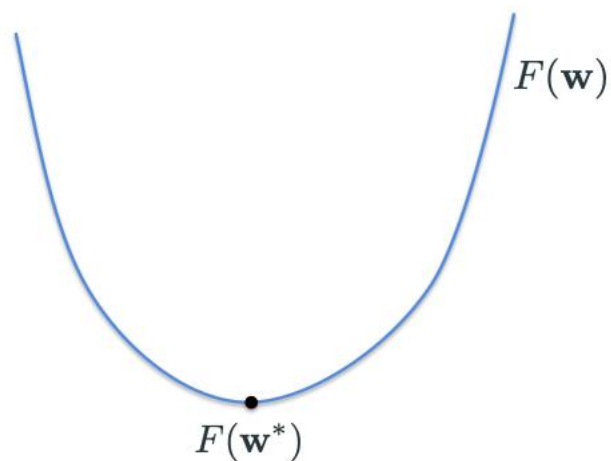


需要结合系统感知的研究方法，融合优化理论与网络/调度技术

随机梯度下降法(SGD)

$$\min_{\mathbf{w}} F(\mathbf{w}) = \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \nabla (y_i - \mathbf{w}^T \mathbf{x})^2$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla F(\mathbf{w}_t)$$

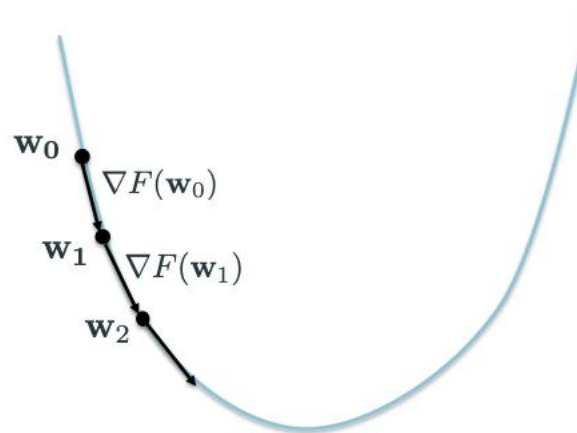
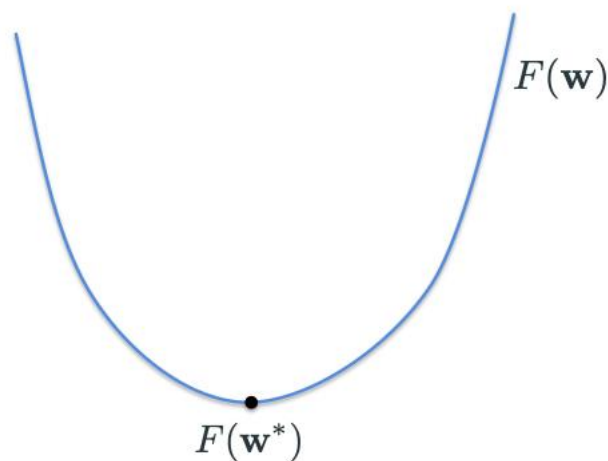


- 在机器学习与优化的核心所在。例如，反向传播算法就是随机梯度下降的一个特例
- 虽然具有内在顺序性，但需要在分布式环境中实现它

随机梯度下降法(SGD)

$$\min_{\mathbf{w}} F(\mathbf{w}) = \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \nabla (y_i - \mathbf{w}^T \mathbf{x})^2$$

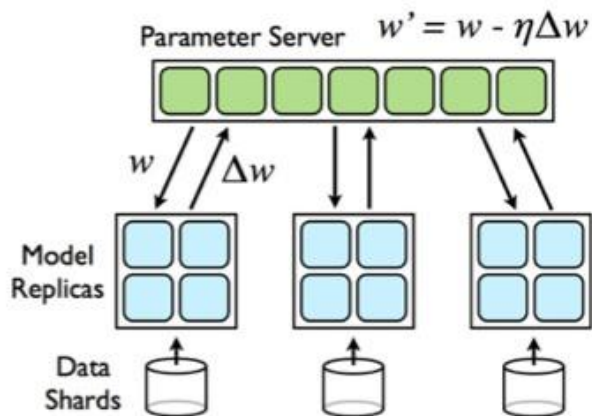
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla F(\mathbf{w}_t)$$



本节讨论机器学习计算模式

- 批量与小批量随机梯度下降
- 动量法、适应及方差缩减方法
- 随机梯度下降的误差收敛分析

分布式随机梯度下降



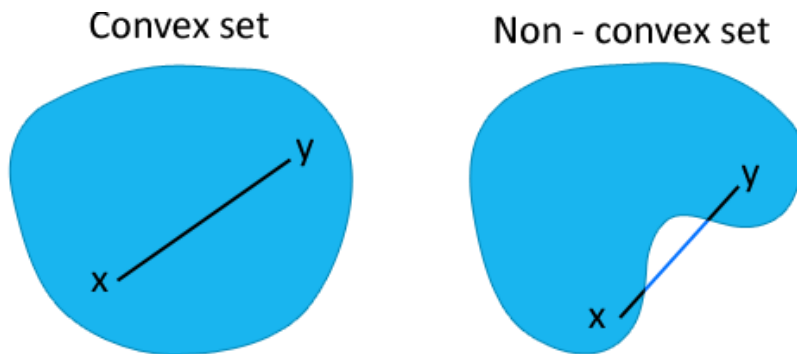
数据集被分割至各节点，其计算的梯度通过不同方式聚合。梯度/模型有如下聚合算法：

- 同步式随机梯度下降
- 异步式随机梯度下降
- 弹性平均法
- 本地更新随机梯度下降
- 量化/稀疏化随机梯度下降

并从误差收敛速度和单次迭代运行时长的角度分析其性能

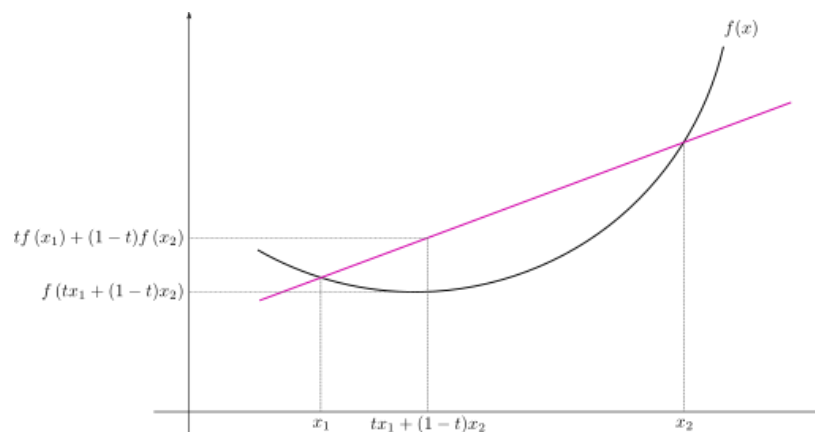
凸性

- 凸集定义: 集合 $X \subseteq \mathbb{R}^d$ 满足 $\forall x, y \in X, t \in [0, 1] tx + (1-t)y \in X$ 即线段连接性



- 凸函数定义: 函数 $f: \mathbb{R}^d \rightarrow \mathbb{R}$ 满足 Jensen 不等式

$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$ 对于所有 $x, y \in \text{定义域} f$, 且 $t \in [0, 1]$ 。



梯度

梯度

考虑一个多元函数 $f: \mathbb{R}^d \rightarrow \mathbb{R}$, 其梯度为:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \quad [\nabla f]_i = \frac{\partial f}{\partial x_i} \quad \forall i \in \{1, 2, \dots, d\}$$

$\nabla f(\mathbf{x})$ 指向从 $\mathbf{x}$ 出发最陡峭上升方向。 示例

- 问题: 求函数 $f(x_1, x_2) = x^2 + 3x_1x_2$ 的梯度。
- 答案: $\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 + 3x_2 \\ 3x_1 \end{bmatrix}$

海森矩阵

海森矩阵

函数 f 的海森矩阵: $\mathbb{R}^d \rightarrow \mathbb{R}$ 定义为:

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \quad [\mathbf{H}_f]_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

示例

- 问:求函数 $f(x_1, x_2) = x^2 + 3x_1x_2$ 的海森矩阵
- 答: $H = \begin{bmatrix} 2 & 3 \\ 3 & 0 \end{bmatrix}$

克莱罗定理

若函数 $f: \mathbb{R}^d \rightarrow \mathbb{R}$ 的二阶偏导数在点 $\mathbf{x}$ 处连续, 则

$$\frac{\partial^2 f}{\partial x_i \partial x_j}(\mathbf{x}) = \frac{\partial^2 f}{\partial x_j \partial x_i}(\mathbf{x}), \quad \forall i, j \in \{1, \dots, d\},$$

在此情况下该海森矩阵是对称的 $[H]_{ij}(\mathbf{x}) = [H]_{ji}(\mathbf{x})$ 。

凸函数的海森矩阵是半正定的, 即

$$\mathbf{x}^T H \mathbf{x} \geq 0 \text{ 对所有 } \mathbf{x} \in \mathbb{R}^d \quad (1)$$

H 是半正定的当且仅当其所有特征值均为非负

优化凸目标函数

- 考虑一个凸目标函数 $f(\mathbf{x}): \mathbb{R}^d \rightarrow \mathbb{R}$, 其依赖于参数向量 $\mathbf{x} \in \mathbb{R}^d$ 。
- 的目标是找到 $\mathbf{x}^*$ 以最小化该函数 即

$$\mathbf{x}^* = \text{最小化 } f(\mathbf{x}) \quad (2)$$

- 如何找到 $\mathbf{x}^*$?
- 寻找 $f(\mathbf{x})$ 的驻点, 即满足 $\nabla f(\mathbf{x}^*) = 0$
- 由于 $f(\mathbf{x})$ 是凸函数, 其驻点即为最小化点

示例

- 问题: 求使 $f(x_1, x_2) = (x_1 - 3)^2 + 2(x_2 - 2)^2$ 的 $\mathbf{x}$
- 解: Ans: $\nabla f(\mathbf{x}) = \begin{bmatrix} 2(x_1 - 3) \\ 4(x_2 - 2) \end{bmatrix} = 0 \Rightarrow$
 $(x_1^*, x_2^*) = (3, 2)$

梯度下降法(GD)

- 在许多问题中, 直接求解满足 $\nabla f(\mathbf{x}) = 0$ *较为困难
- 梯度下降(GD)是一种迭代算法, 通过沿负梯度方向 $-\nabla f(\mathbf{x})$ 逐步逼近极小值点。从随机初始点 $\mathbf{x}_0$ 出发:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t) \quad (3)$$

其中 η 为学习率且 $\eta > 0$ 。

直观理解: 如同雾中下山者, 通过感知坡度沿最陡方向逐步移动至谷底

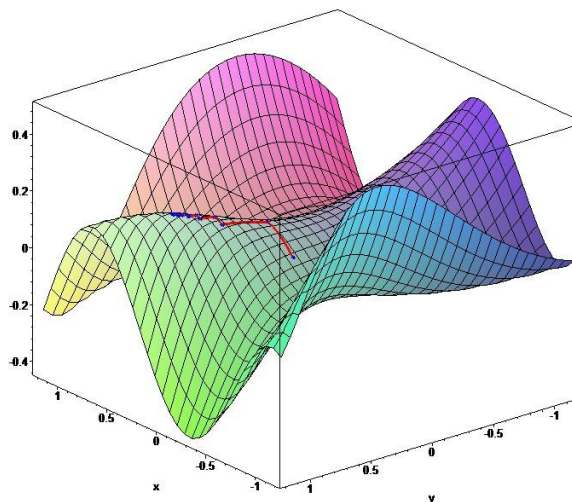
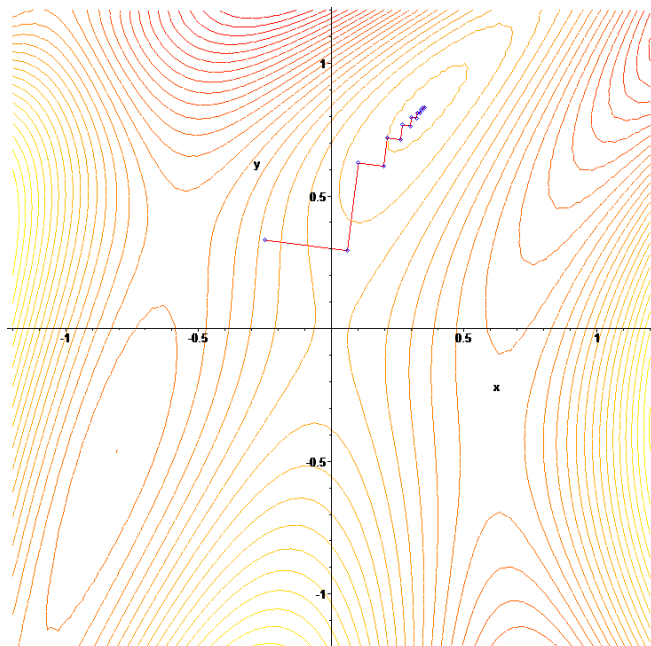
梯度下降法(GD)

- 梯度下降 (GD) 从一个随机初始点 $\mathbf{x}_0$ 出发, 并按照以下方式更新 $\mathbf{x}$:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}) \quad (4)$$

其中 η 为学习率且 $\eta > 0$ 。

- 对于凸函数 F 及足够小的学习率 η , GD保证能收敛到最优解 $\mathbf{x}^*$ (更多内容后续详述)
- 但对于非凸函数, 可能陷入局部极小值或鞍点。



概念检查练习

- 问题：写出函数 $f(\mathbf{x}) = 2x_1^2 + x_2^2$ 梯度下降更新规则
- 答案： $\nabla f(\mathbf{x}) = [4x_1, 2x_2]^T$

$$x_1^{(t+1)} = x_1^{(t)} - \eta(4x_1^{(t)})$$

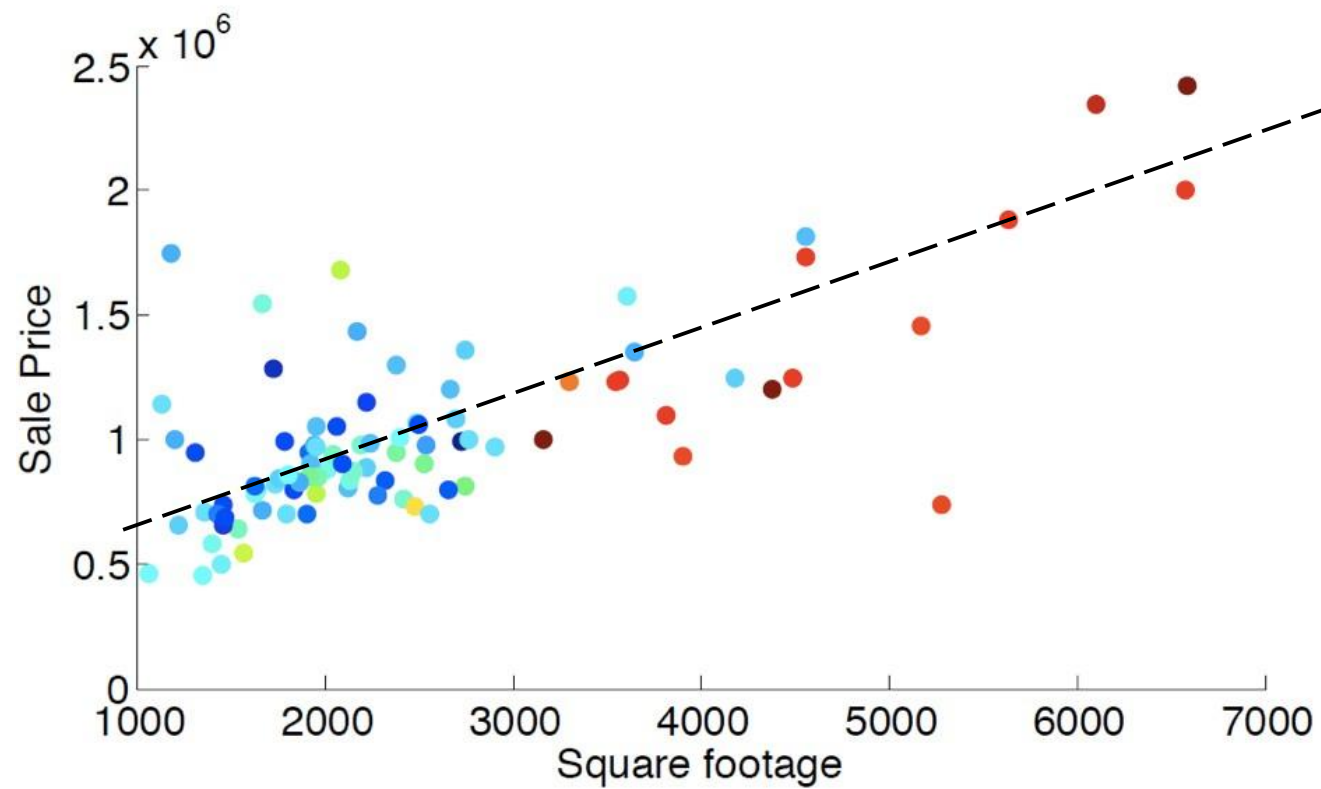
$$x_2^{(t+1)} = x_2^{(t)} - \eta(2x_2^{(t)})$$

目录

Contents

- 1 分布式机器学习概述
- 2 **SGD与mini-SGD**
- 3 SGD的变体
- 4 目标函数的性质
- 5 梯度下降的收敛性分析

示例：预测房价



售价 $\approx$ 每平方英尺单价 $\times$ 面积+ 固定成本

线性回归

设置:

- **输入:** $\mathbf{x} \in \mathbb{R}^d$ (协变量、预测因子、特征等)
- **输出:** $y \in \mathbb{R}$ (响应、目标、结果等)
- **模型:** $f: \mathbf{x} \rightarrow y$, 其中 $f(\mathbf{x}) = w_0 + \sum_{d=1}^D w_d x_d = w_0 + \mathbf{w}^\top \mathbf{x}$.
 - $\mathbf{w} = [w_1 w_2 \cdots w_D]^\top$: 权重, 参数, 或参数向量
 - w_0 称为偏置.
 - 有时, 也称 $\mathbf{w} = [w_0 \ w_1 \ w_2 \ \cdots \ w_D]^\top$
- **训练数据:** $D = \{(\mathbf{x}_n, y_n), n = 1, 2, \dots, N\}$

最小化经验风险函数, 即线性回归中的残差平方和:

当 $\mathbf{x}$ 为 D 维时的最小均方

$F(\mathbf{w})$ 的矩阵形式:

$$F(\mathbf{w}) = \sum_n [y_n - (w_0 + \sum_d w_d x_{nd})]^2 = \sum_n [y_n - \mathbf{w}^\top \mathbf{x}_n]^2,$$

此处通过扩展重新定义了部分变量 (增广处理)

$$\mathbf{x} \leftarrow [1 x_1 x_2 \dots x_D]^\top, \quad \mathbf{w} \leftarrow [w_0 w_1 w_2 \dots w_D]^\top$$

设计矩阵与目标向量:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times (D+1)}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

紧凑表达式:

$$F(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} \right\} + \text{const}$$

最小二乘解

紧凑表达式 $F(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \left\{ \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} \right\} + \text{const}$

线性与二次函数的梯度

- $\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \mathbf{b}$
- $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A}\mathbf{x}$ (对称矩阵 $\mathbf{A}$)

正规方程

$$\nabla_{\mathbf{w}} F(\mathbf{w}) = 2\mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{X}^\top \mathbf{y} = 0$$

得到最小均方(LMS)解

$$\mathbf{w}^{LMS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

计算复杂度

求解过程中的瓶颈是什么？

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

需要多少运算量？

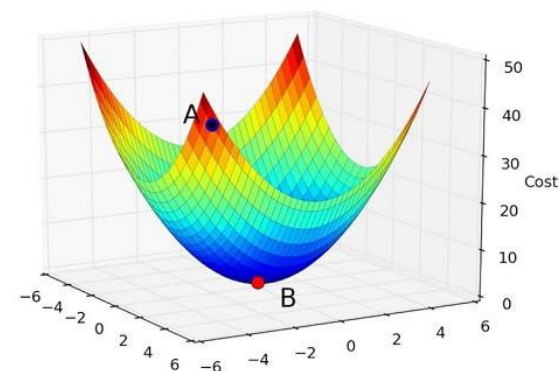
- 矩阵乘法 $\mathbf{X}^\top \mathbf{X}$ 的复杂度为 $O(ND^2)$
- 矩阵求逆 $\mathbf{X}^\top \mathbf{X}$ 的复杂度为 $O(D^3)$ (如使用Gauss-Jordan消元法)或 $O(D^{2.373})$ (最新理论进展)
- 矩阵乘法 $\mathbf{X}^\top \mathbf{y}$ 的复杂度为 $O(ND)$
- $(\mathbf{X}^\top \mathbf{X})^{-1}$ 复杂度为 $O(D^2)$

总复杂度为 $O(ND^2) + O(D^3)$ ——对于非常大的D或N不实用

替代方法：批量梯度下降

(批量)梯度下降

- 初始化 $\mathbf{w}$ 为 $\mathbf{w}^{(0)}$ (如随机初始化); 设 $t=0$; 选择 $\eta>0$
- 循环至收敛
 1. 计算梯度
$$\nabla F(\mathbf{w}) = \mathbf{X}^\top (\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$$
 2. 更新参数
$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla F(\mathbf{w}^{(t)})$$
 3. $t \leftarrow t+1$



每次迭代的复杂度是多少?

$O(ND)$

为何此法有效?

若梯度下降收敛, 其解与矩阵求逆法相同

这是因为 $F(\mathbf{w})$ 是其参数 $\mathbf{w}$ 的凸函数

F 的海森矩阵

$$\begin{aligned} F(\mathbf{w}) &= \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 (\mathbf{X}^\top \mathbf{y})^\top \mathbf{w} + \text{const} \\ \Rightarrow \frac{\partial^2 F(\mathbf{w})}{\partial \mathbf{w} \mathbf{w}^\top} &= 2 \mathbf{X}^\top \mathbf{X} \end{aligned}$$

$\mathbf{X}^\top \mathbf{X}$ 是半正定的, 因为对于任意 $\mathbf{v}$

$$\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = \|\mathbf{X}^\top \mathbf{v}\|_2^2 \geq 0$$

随机梯度下降法(SGD)

威德罗-霍夫规则：每次使用一个样本更新参数

- 初始化 $\mathbf{w}$ 为某个 $\mathbf{w}^{(0)}$ ；设 $t=0$ ；选择 $\eta>0$
- 循环至收敛

1. 随机选取一个训练样本 $\mathbf{x}_t$
2. 计算其对梯度的贡献

$$\mathbf{g}_t = (\mathbf{x}_t^\top \mathbf{w}^{(t)} - y_t) \mathbf{x}_t$$

3. 更新参数

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \mathbf{g}_t$$

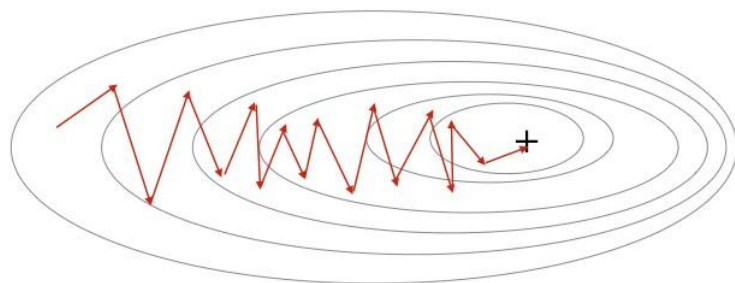
4. $t \leftarrow t+1$

每次迭代的复杂度与梯度下降法相比如何？

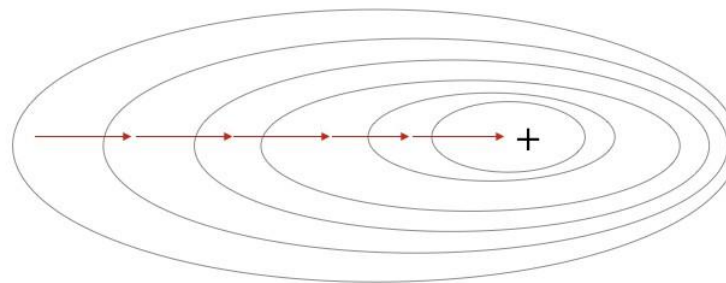
- 梯度下降法为 $O(ND)$ ，而SGD为 $O(D)$

SGD对比批量梯度下降

Stochastic Gradient Descent

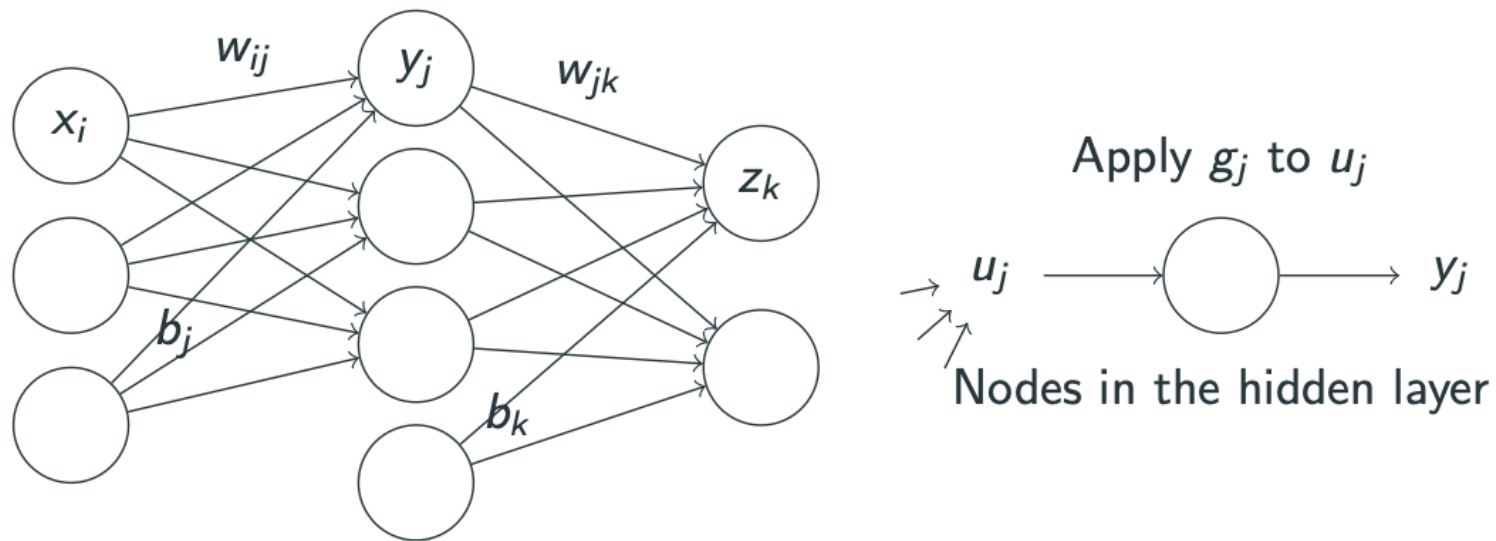


Gradient Descent



- SGD将每次迭代的复杂度从 $O(ND)$ 降低到 $O(D)$
- 但其存在噪声且可能需要更长时间才能收敛

多层神经网络



- w_{ij} : **权重** 连接节点 i 在层 $(\ell-1)$ 到节点 j 在层 ℓ 。
- b_j, b_k : **偏置** 用于节点 j 和 k 分别。
- u_j, u_k : **输入** 到节点 j 和 k (其中 $u_j = b_j + \sum_i x_i w_{ij}$)。
- g_j, g_k : **激活函数** 用于节点 j (应用于 u_j) 和节点 k 。
- $y_j = g_j(u_j), z_k = g_k(u_k)$: **输出/激活** 的节点 j 和 k 。
- t_k : **目标值** 对于节点 k 在输出层。

学习参数

如何学习这些参数?

- 选择正确的损失函数

- 回归:最小二乘损失

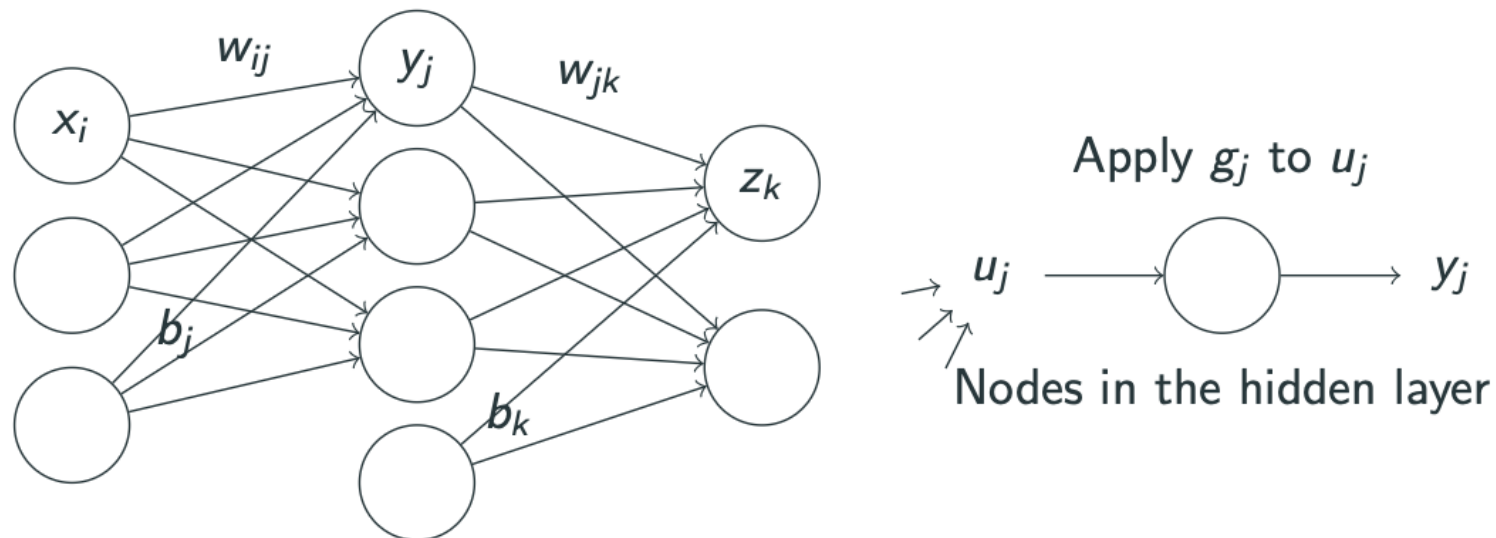
$$\min \sum_n (f(\mathbf{x}_n) - t_n)^2$$

- 分类:交叉熵损失

$$\min - \sum_n \sum_k t_{nk} \log f_k(\mathbf{x}_n) + (1 - t_{nk}) \log(1 - f_k(\mathbf{x}_n))$$

- 由于 f (神经网络输出) 是 $\mathbf{x}_n$ 的复杂函数, 因此这是一个困难的优化问题
 - 通常采用随机梯度下降法
 - 应用了多种优化技巧

随机梯度下降



- 随机选取一个数据点 $(\mathbf{x}_n, t_n)$
- 仅使用该数据点计算梯度，例如：

$$\Delta = \frac{\partial [f(\mathbf{x}_n) - t_n]^2}{\partial w}$$

- 更新参数: $\mathbf{w} \leftarrow \mathbf{w} - \eta \Delta$
- 迭代该过程直至满足预设的停止条件

更新参数值

反向传播误差。给定参数 w, b :

- 步骤1: 前向传播以根据输入 (“前馈信号”) 找出 z_k
- 步骤2: 通过比较预测输出 z_k 与真实值 t_k , 计算输出误差 E 。
- 步骤3: 通过将 E 按相关激活函数的梯度和前一层权重加权, 进行误差反向传播

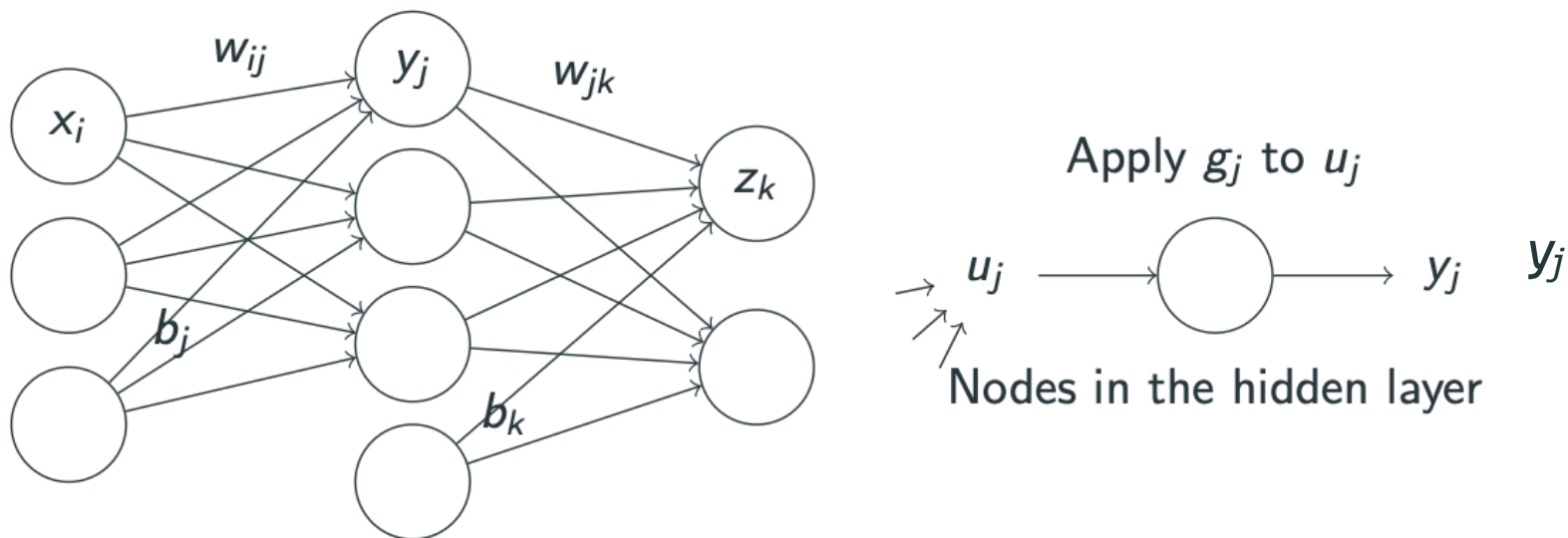
- 步骤4: 计算参数 w 和 b 的梯度 $\frac{\partial E}{\partial w}$ and $\frac{\partial E}{\partial b}$

基于反向传播的误差信号和来自输入的前馈信号, 逐层计算梯度

- 步骤5: 使用计算得到的梯度更新参数

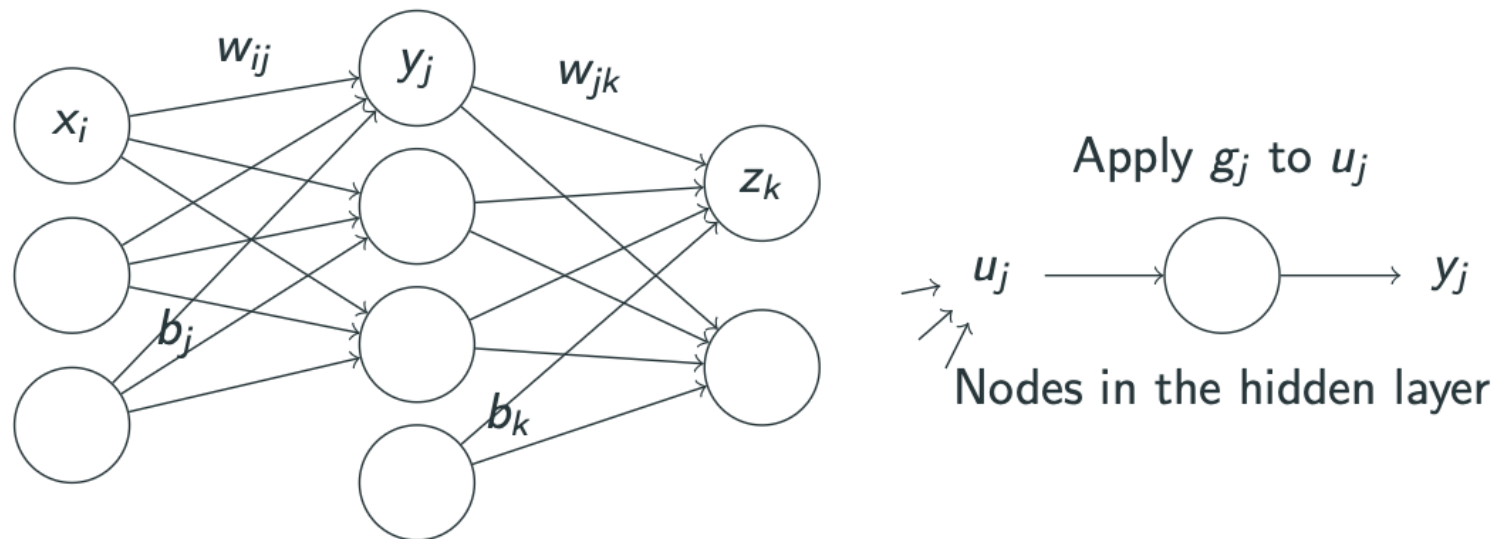
$$w \leftarrow w - \eta \frac{\partial E}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial E}{\partial b} \quad \text{其中} \eta \text{为步长}$$

示例说明示例



- w_{ij} : **权重**连接节点 i 在层 $(\ell-1)$ 到节点 j 在层 ℓ 。
- b_j, b_k : 节点 j 和 k 的**偏置**。
- u_j : **输入**到节点 j 和 k (其中 $u_j = b_j + \sum_i x_i w_{ij}$)。
- g_j, g_k : **激活函数**用于节点 j (应用于 u_j) 和节点 k 。
- $y_j = g_j(u_j), z_k = g_k(u_k)$: **输出/激活**的节点 j 和 k 。
- t_k : **目标值**对于节点 k 在输出层。

示例 (步骤1与2)



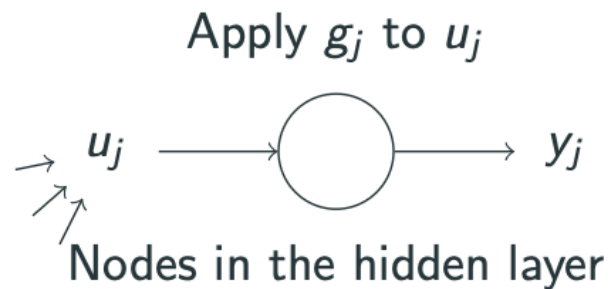
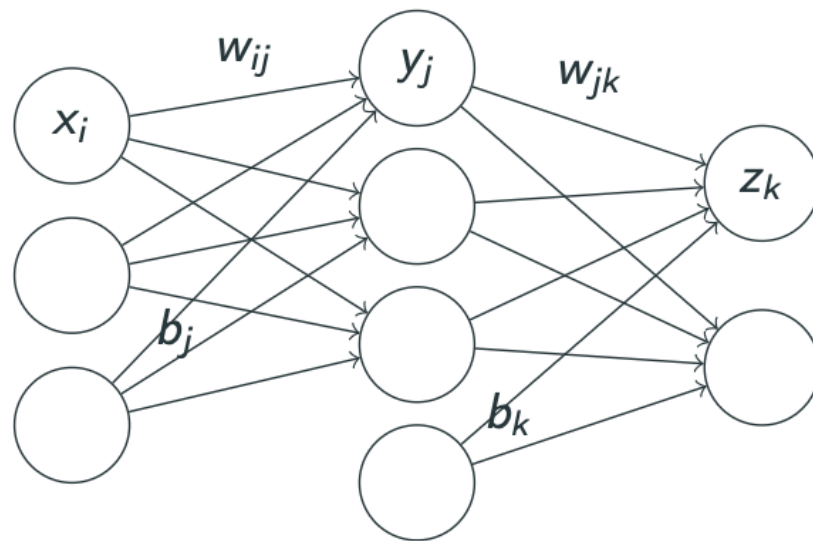
- 步骤1: 对每个输出 z_k 进行前向传播。

$$z_k = g_k(u_k) = g_k(b_k + \sum_j y_j w_{jk}) = g_k(b_k + \sum_j g_j(b_j + \sum_i x_i w_{ij}) w_{jk})$$

步骤2: 计算误差。假设误差函数为目标值与网络输出之差的平方和, 即目标值 t_k 与网络输出 z_k 的差值平方和

网络输出 z_k 的误差 $E = \frac{1}{2} \sum_{k \in K} (z_k - t_k)^2$.

示例（步骤3，输出层）



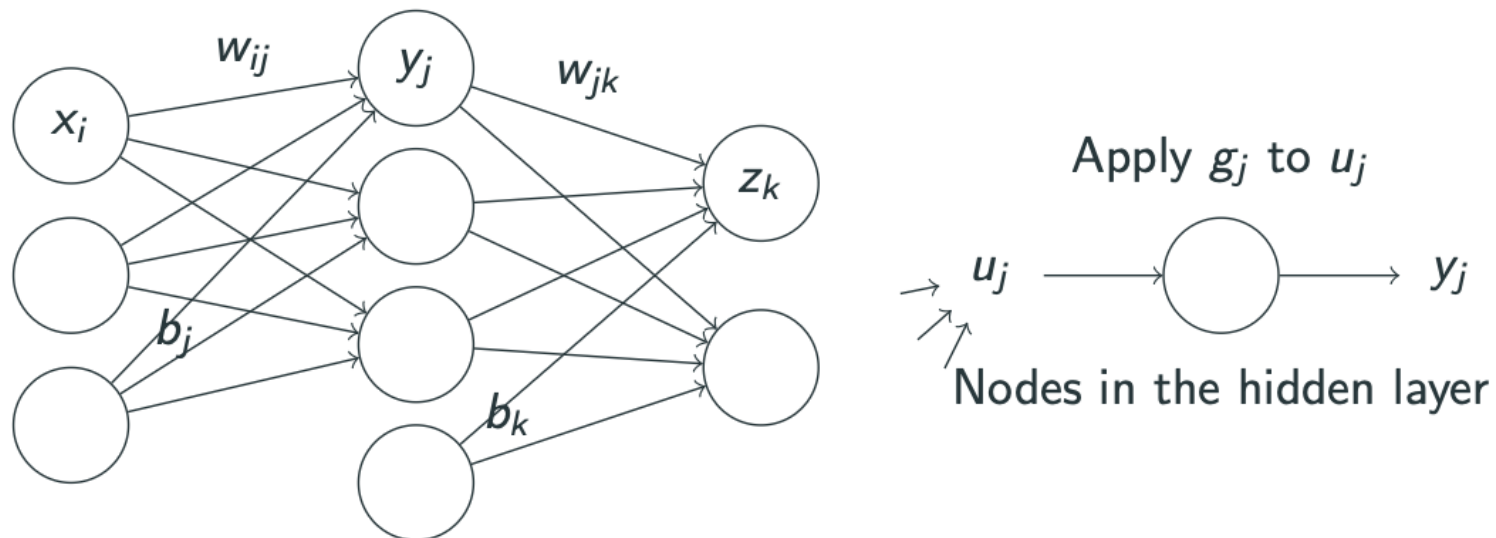
步骤3: 反向传播误差。从输出层权重 w_{jk} 开始,

$$E = \frac{1}{2} \sum_{k \in K} (z_k - t_k)^2, u_k = b_k + \sum_j w_{jk} y_j:$$

$$\begin{aligned} \frac{\partial E}{\partial w_{jk}} &= \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial u_k} \frac{\partial u_k}{\partial w_{jk}} = (z_k - t_k) \frac{\partial z_k}{\partial u_k} \frac{\partial u_k}{\partial w_{jk}} \\ &= (z_k - t_k) g'_k(u_k) \frac{\partial}{\partial w_{jk}} u_k = (z_k - t_k) g'_k(u_k) y_j = \delta_k y_j \end{aligned}$$

其中 $\delta_k = (z_k - t_k) g'_k(u_k)$ 被称为 u_k 的误差

示例 (步骤3, 隐藏层)



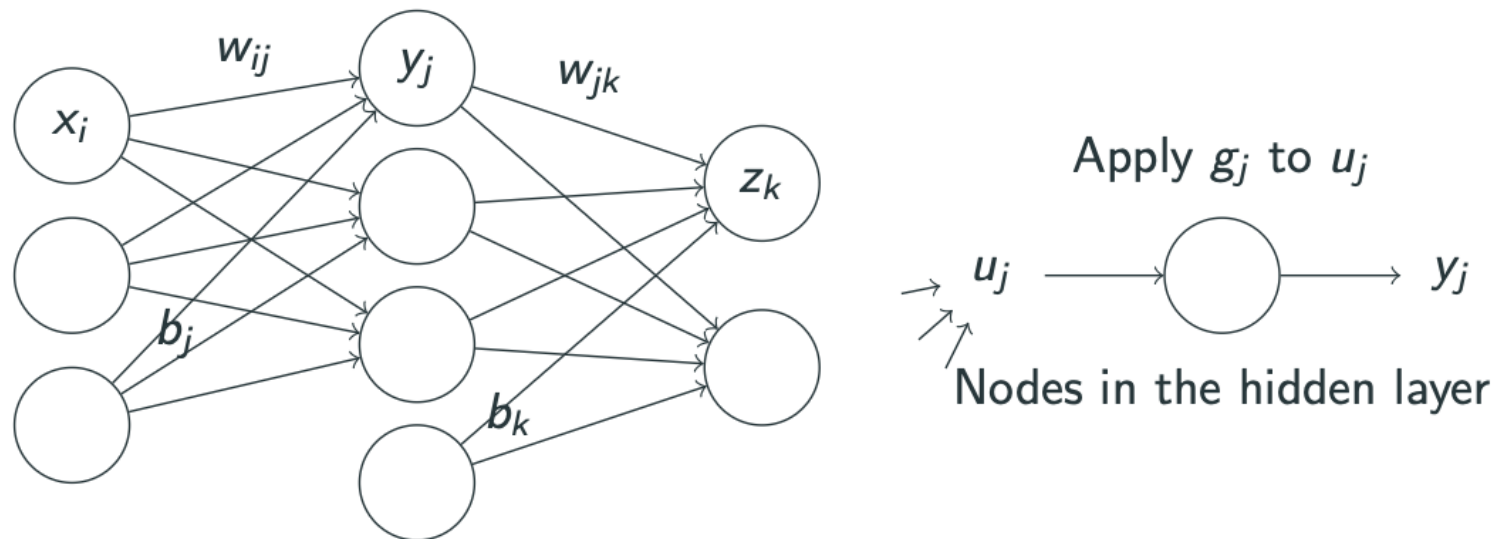
步骤3 (续): 现在考虑隐藏层中的 w_{ij} , 回顾

$$u_j = b_j + \sum_i x_i w_{ij}, \quad u_k = b_k + \sum_j g_j(u_j) w_{jk}, \quad z_k = g_k(u_k):$$

$$\begin{aligned} \frac{\partial E}{\partial w_{ij}} &= \sum_{k \in K} \frac{\partial E}{\partial u_k} \frac{\partial u_k}{\partial y_j} \frac{\partial y_j}{\partial u_j} \frac{\partial u_j}{\partial w_{ij}} = \sum_{k \in K} \delta_k w_{jk} \frac{\partial y_j}{\partial u_j} \frac{\partial u_j}{\partial w_{ij}} \\ &= \sum_{k \in K} \delta_k w_{jk} g'_j(u_j) x_i = \delta_j x_i \end{aligned}$$

其中代入 $\delta_j = g'_j(u_j) \sum_{k \in K} (z_k - t_k) g'_k(u_k) w_{jk}$

示例 (步骤3和4)

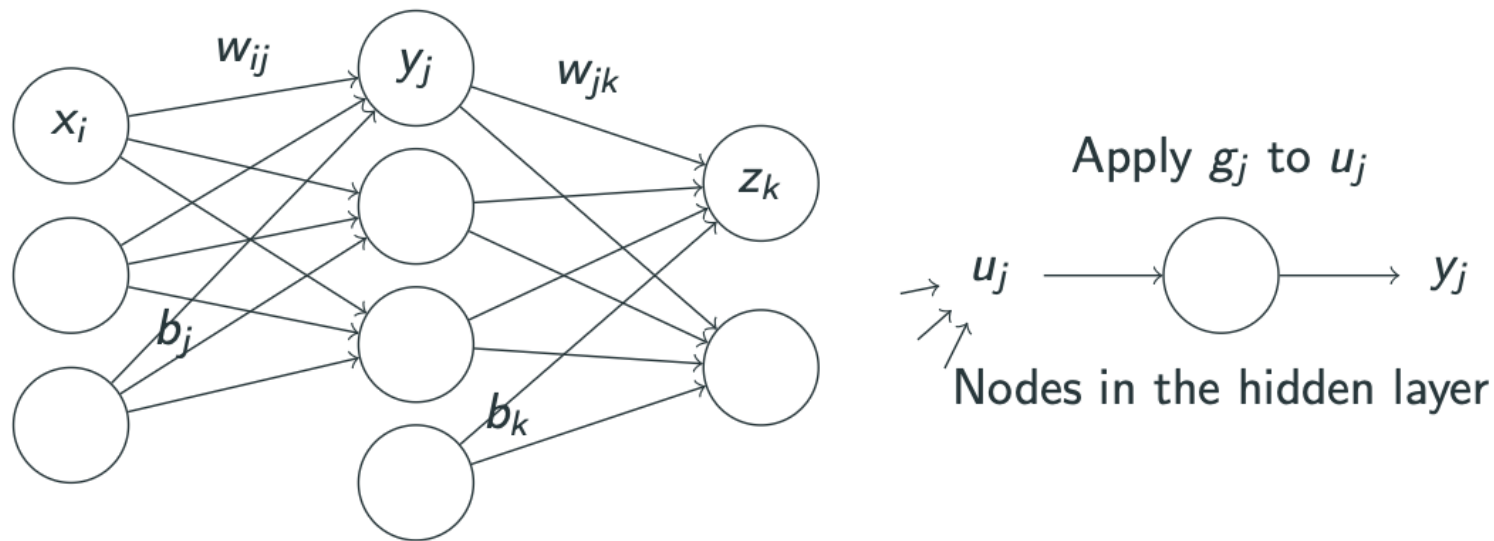


- 步骤3 (续) : 同样发现 $\frac{\partial E}{\partial b_k} = \delta_k$, $\frac{\partial E}{\partial b_j} = \delta_j$.
- 步骤4: 计算梯度。已发现 $\frac{\partial E}{\partial w_{ij}} = \delta_j x_i$ and $\frac{\partial E}{\partial w_{jk}} = \delta_k y_j$.

其中 $\delta_k = (z_k - t_k)g'_k(u_k)$, $\delta_j = g'_j(u_j) \sum_{k \in K} (z_k - t_k)g'_k(u_k)w_{jk}$.

现在既然已知对于特定参数集 w 、 b 的 z_k 、 y_j 、 x_i 、 u_k 及 u_j , 可利用这些表达式来每次迭代计算梯度并更新

示例 (步骤4与5)



- 步骤4: 计算梯度。已发现 $\frac{\partial E}{\partial b_k} = \delta_k, \frac{\partial E}{\partial b_j} = \delta_j$.

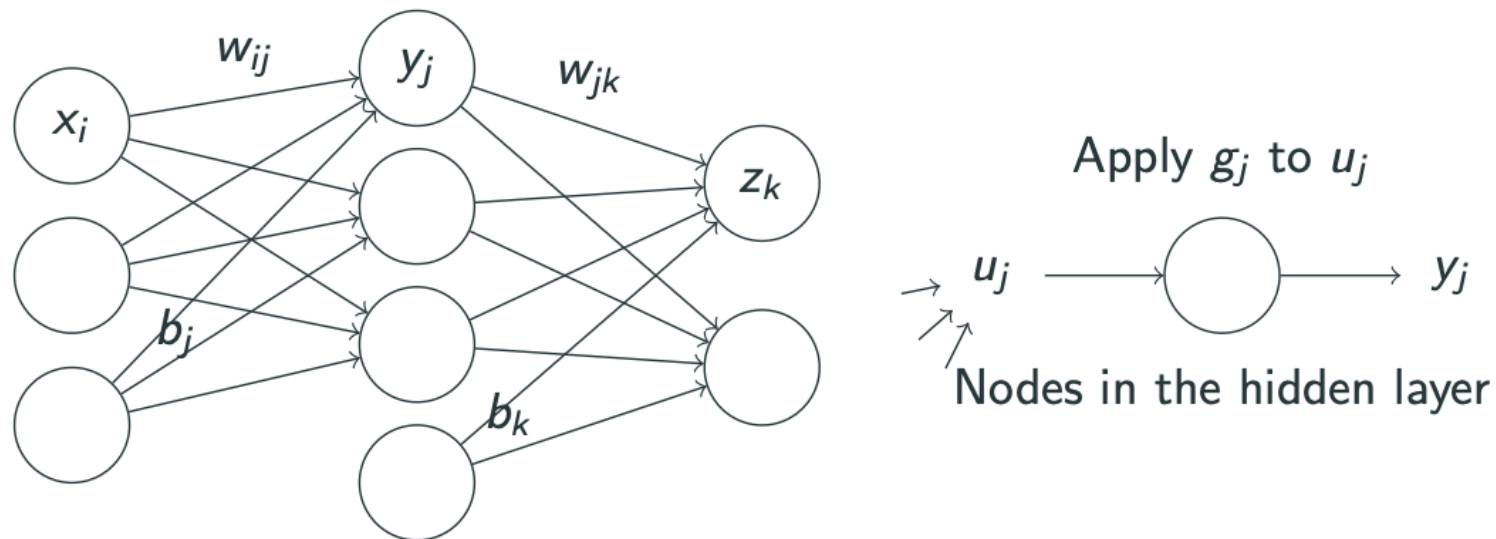
$$\frac{\partial E}{\partial w_{ij}} = \delta_j x_i \text{ and } \frac{\partial E}{\partial w_{jk}} = \delta_k y_j.$$

其中, $\delta_k = (z_k - t_k)g'_k(u_k)$, $\delta_j = g'_j(u_j) \sum_{k \in K} (z_k - t_k)g'_k(u_k)w_{jk}$.

- 步骤5: 以学习率 η 更新权重与偏置

$$w_{jk} \leftarrow w_{jk} - \eta \frac{\partial E}{\partial w_{jk}} \text{ and } w_{ij} \leftarrow w_{ij} - \eta \frac{\partial E}{\partial w_{ij}}$$

高层流程：可扩展至更多隐藏层



最终层

- 各输出误差为 $z_k - t_k$ 。
- 最终层输入 u_k 的误差为 $\delta_k = g'_k(u_k)(z_k - t_k)$

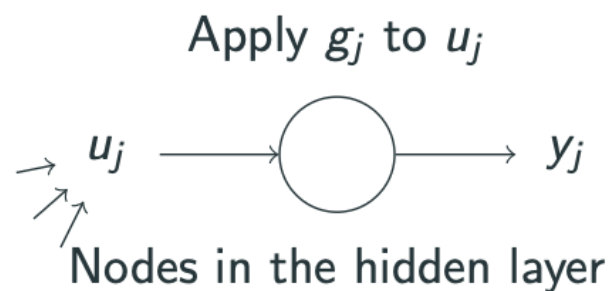
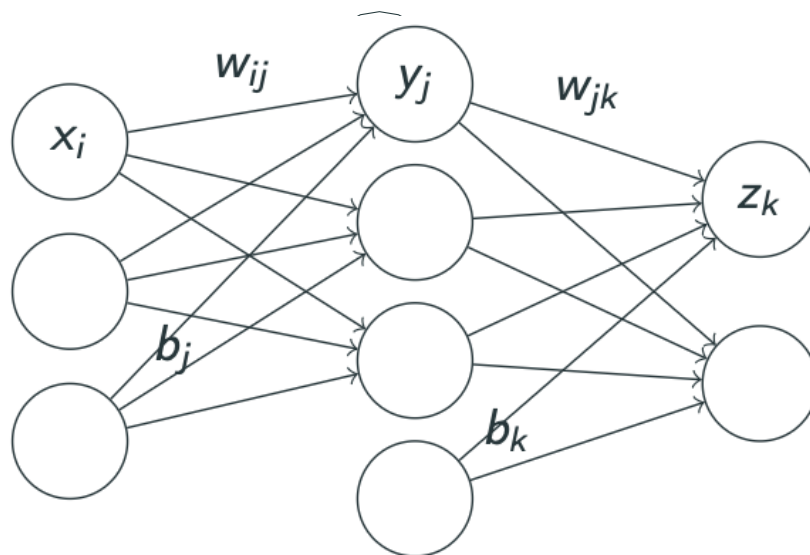
隐藏层

- 输出 y 的误差为 $\sum_{k \in K} \delta_k w_{jk}$ 。
- 输入 u_j 的隐藏层误差为 δ_j : $\delta_j = g'_j(u_j) \sum_{k \in K} \delta_k w_{jk}$

关于 w_{ij} 的梯度为 $x_i \delta_j$ 。

向量化实现

比逐层遍历所有神经元实现循环快得多

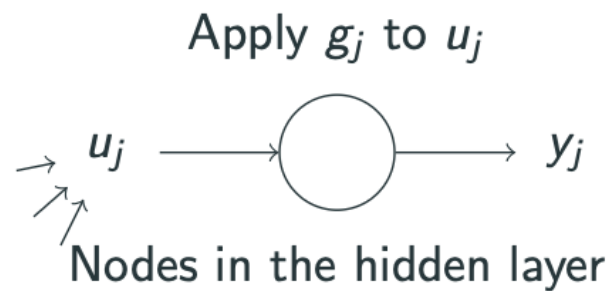
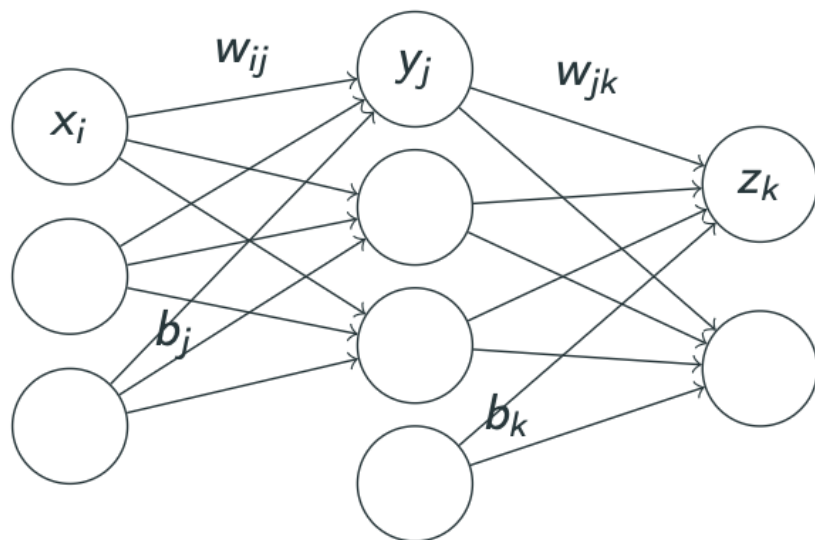


前向传播

- 将第 $l-1$ 层与第 l 层间的权重表示为矩阵 $\mathbf{W}^{(l)}$
- 第 $l-1$ 层的输出为行向量 $\mathbf{y}^{(l-1)}$ ，则有 $\mathbf{u}^{(l)} = \mathbf{y}^{(l-1)} \mathbf{W}^{(l)}$ 。
- 第 l 层的输出为行向量 $\mathbf{y}^{(l)} = g(\mathbf{u}^{(l)})$ 。

向量化实现

比逐层遍历所有神经元实现循环快得多



反向传播

- 对于每一层 l , 根据最终误差找出其输入 $u^{(l)}$ 的误差向量 $\Delta^{(l)}$
- 利用 $\Delta^{(l)}$ 更新权重 $\mathbf{W}^{(l)}$
- 递归地用 $\Delta^{(l)}$ 表示 $\Delta^{(l-1)}$

小批量SGD

- 回顾线性回归中使用的经验风险损失函数 $F(\mathbf{w}) = \sum_n [y_n - \mathbf{w}^\top \mathbf{x}_n]^2$,

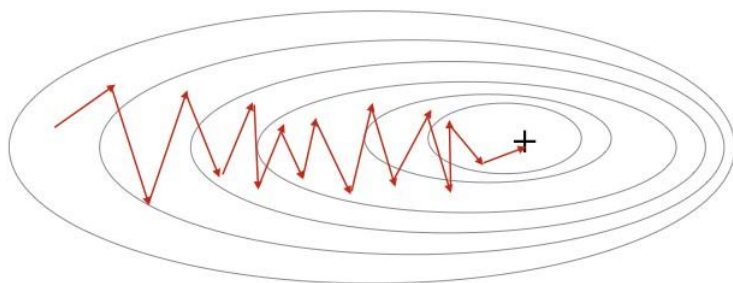
- 对于大型训练数据集（大 N ），逐点计算梯度开销巨大。例如，单次迭代中批量梯度为

$$\nabla F(\mathbf{w}) = \sum_{n=1}^N 2(\mathbf{w}^\top \mathbf{x}_n - y_n) \mathbf{x}_n$$

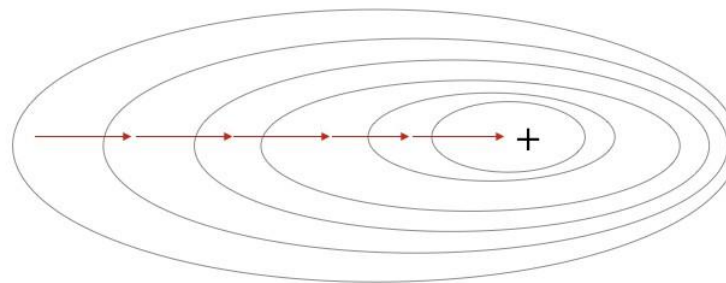
- 因此采用随机梯度下降（SGD），即随机选取数据点 $\mathbf{x}_n$ ，以 $F(\mathbf{w}) = [y_n - \mathbf{w}^\top \mathbf{x}_n]^2$ 替代完整求和

SGD对比批量梯度下降

Stochastic Gradient Descent



Gradient Descent



- SGD将每次迭代的复杂度从 $O(ND)$ 降低到 $O(D)$
- 但其存在噪声且可能需要更长时间才能收敛

小批量SGD

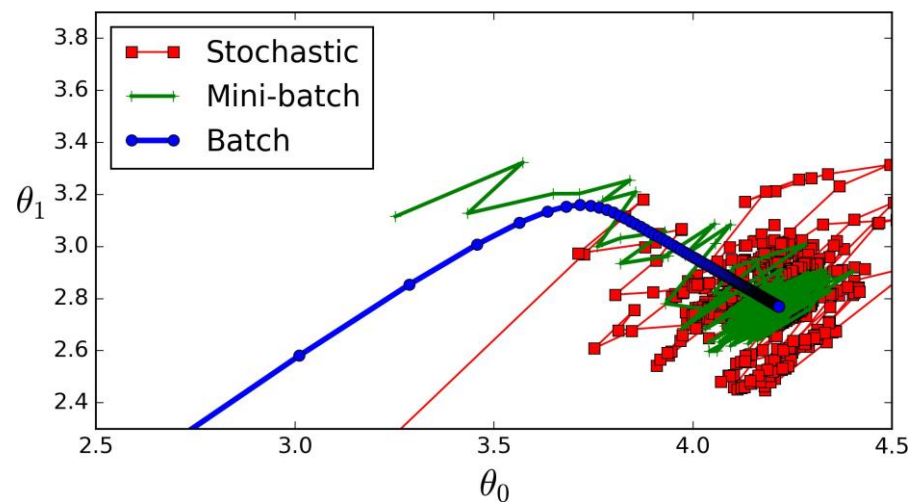
- 小批量SGD是这两种极端情况的折中
- 每次迭代从 N 个样本中随机选取 b 个样本构成集合 S 训练样本并计算

$$F(\mathbf{w}) = \sum_{n \in S} [y_n - \mathbf{w}^\top \mathbf{x}_n]^2,$$

- 较小的 b 节省单次迭代计算成本，但会增加梯度噪声，导致误差表现更差
- 较大的批次大小能降低梯度噪声并提供更优的误差收敛性，但会提高计算成本每次迭代的开销

如何选择小批量大小

- 小型训练数据集——使用批量梯度下降法 ($b = N$)
- 大型训练数据集——典型批次大小为64、128、256...等，只要能放入CPU/GPU内存即可
- 小批量大小是另一个超参数需要进行调优的



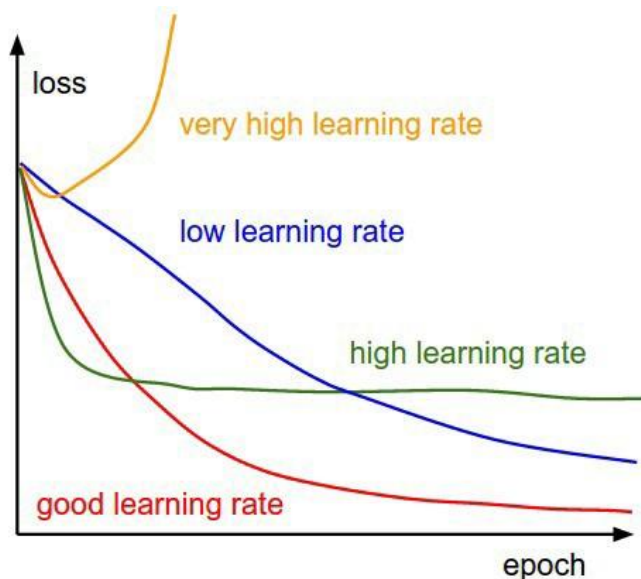
图片来源: <https://github.com/buomsoo-kim/Machine-learning-toolkits-with-python>

学习率

- 随机梯度下降更新规则

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial F}{\partial w^{(t)}} = w^{(t)} - \eta \nabla F(w^{(t)})$$

- 较大的 η ：收敛快但误差平台较高（即曲线平坦部分）
- 较小的 η ：收敛慢但误差平台较低（蓝色曲线最终会低于红色曲线）
- 要兼顾两者优势，可随时间逐步衰减 η



目录

Contents

- 1 分布式机器学习概述
- 2 SGD与mini-SGD
- 3 SGD的变体**
- 4 目标函数的性质
- 5 梯度下降的收敛性分析

SGD变体

以下讨论的参考文献

- SGD方法综述(1)作者: Ruder
<https://arxiv.org/pdf/1609.04747.pdf>
- SGD方法综述(2)作者: Sun等
<https://arxiv.org/pdf/1906.06821.pdf>
- 动量法的可调演示
<https://distill.pub/2017/momentum/>

将讨论以下基础内容:

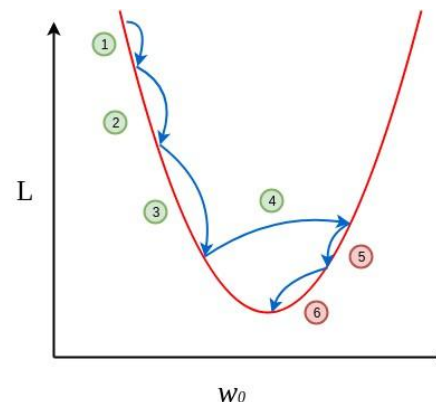
- 动量法原理
- 自适应方法

动量

- 普通SGD的缺点：虽稳定但向最小值收敛的速度通常较慢，类似于以恒定速度移动

$$w^{(t+1)} = w^{(t)} - \eta \nabla E(w^{(t)})$$

- 当 $\nabla E(w^{(t)})$ 较小时，SGD会减速
- 动量SGD：利用运动惯性加速未来步长——若第t+1步的梯度 $\nabla E(w^{(t)})$ 较大，其一部分会被累加至第t+2、t+3...t+n次更新中。
- 类比于沿斜坡滚下重球时速度逐渐加快。



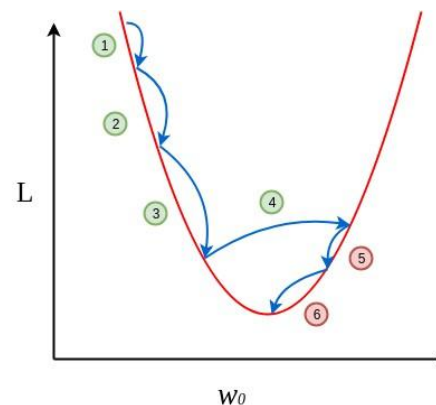
动量

- 动量SGD：通过运动惯性加速SGD的未来步长。
- 正式更新规则如下：

$$\begin{aligned}v^{(t)} &= \gamma v^{(t-1)} + \eta \nabla E(w^{(t)}) \\w^{(t+1)} &= w^{(t)} - v^{(t)}\end{aligned}$$

其中，第 $t + 1$ 次迭代的更新向量 $v^{(t)}$ 包含当前梯度及前次更新向量的一小部分 γ

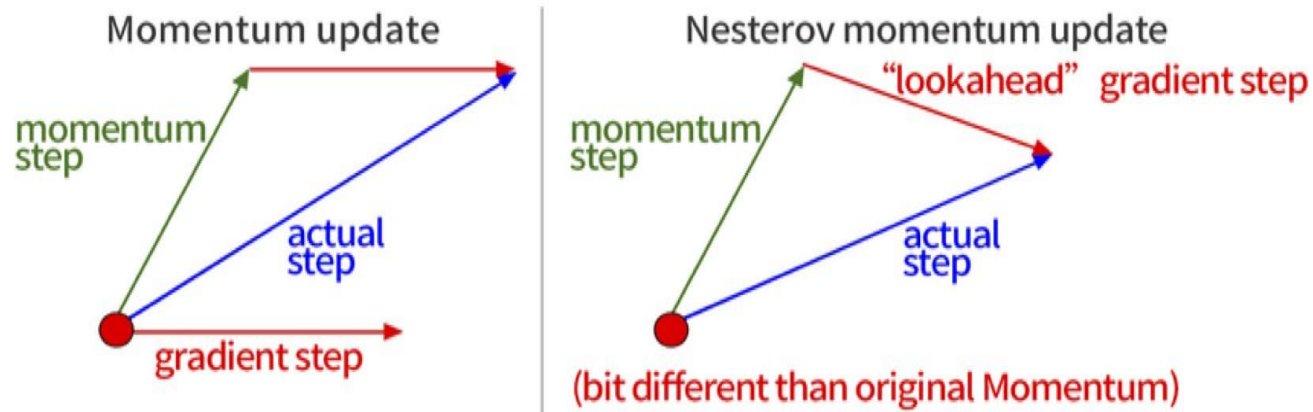
- 动量系数 γ 通常设为0.9



涅斯捷罗夫动量

- 动量可能使球体加速过度，导致其越过最小值
- 涅斯捷罗夫动量通过微调更新规则以预判梯度未来变化

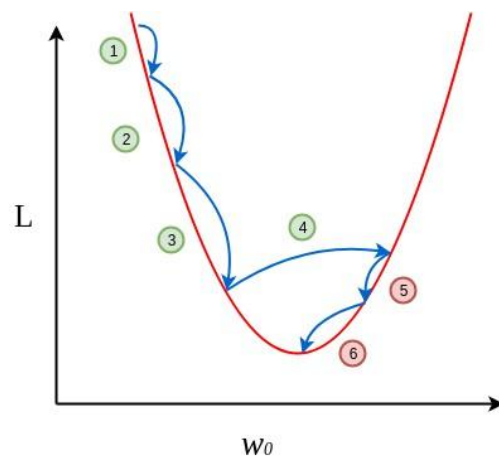
$$\begin{aligned} \mathbf{v}^{(t)} &= \gamma \mathbf{v}^{(t-1)} + \eta \nabla E(\mathbf{w}^{(t)} - \gamma \mathbf{v}^{(t-1)}) \\ \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \mathbf{v}^{(t)} \end{aligned}$$



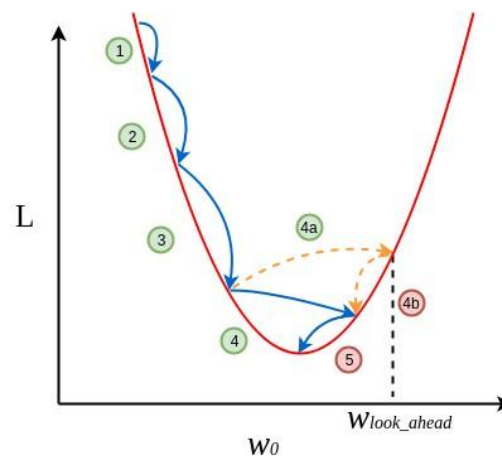
涅斯捷罗夫动量

- 动量可能使球体加速过度，导致其越过最小值
- 涅斯捷罗夫动量通过微调更新规则以预判梯度未来变化

$$\begin{aligned} \mathbf{v}^{(t)} &= \gamma \mathbf{v}^{(t-1)} + \eta \nabla E(\mathbf{w}^{(t)} - \gamma \mathbf{v}^{(t-1)}) \\ \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \mathbf{v}^{(t)} \end{aligned}$$



(a) Momentum-Based Gradient Descent



(b) Nesterov Accelerated Gradient Descent

$$\text{Green Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$$

$$\text{Red Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$$

SGD变体

以下讨论的参考文献

- SGD方法综述(1)作者: Ruder
<https://arxiv.org/pdf/1609.04747.pdf>
- SGD方法综述(2)作者: Sun等
<https://arxiv.org/pdf/1906.06821.pdf>
- 动量法的可调演示
<https://distill.pub/2017/momentum/>

将讨论以下基础内容:

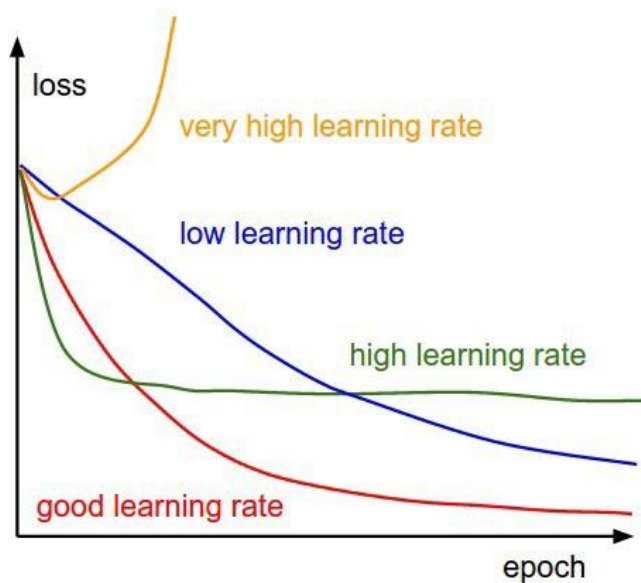
- 动量法原理
- 自适应方法

学习率

- 随机梯度下降更新规则

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial E}{\partial w^{(t)}} = w^{(t)} - \eta \nabla E(w^{(t)})$$

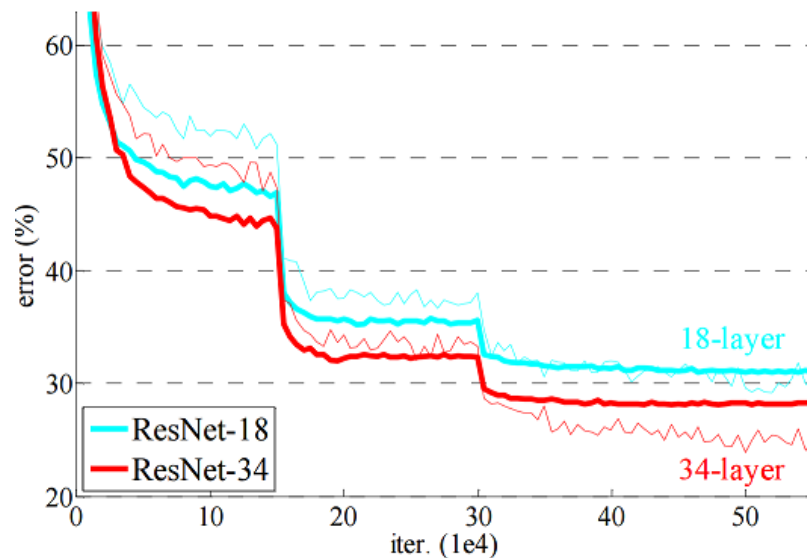
- 较大的 η ：收敛快但误差平台较高（即曲线平坦部分）
- 较小的 η ：收敛慢但误差平台较低（蓝色曲线最终会低于红色曲线）
- 要兼顾两者优势，可随时间逐步衰减 η



如何衰减学习率?

一种常见方法是衰减 η

- 初始设定学习率如 $\eta = 0.1$
- 监测训练损失直至其趋于平缓
- 按固定比例 (如5倍) 降低 η 至新值0.02
- 当曲线再次平缓时重复相同比例的衰减



图片来源: <http://www.bdhammel.com/learning-rates/>

AdaGrad

一种替代方法——AdaGrad[Duchi等人2011]

- 将学习率 η 除以梯度平方和的平方根（截至当前时刻）
- 该缩放因子根据各参数对应的梯度不同而有所差异

$$\mathbf{w}_i^{(t+1)} = \mathbf{w}_i^{(t)} - \frac{\eta}{\sqrt{g_i^{(t)} + \epsilon}} \nabla E(\mathbf{w}_i^{(t)})$$

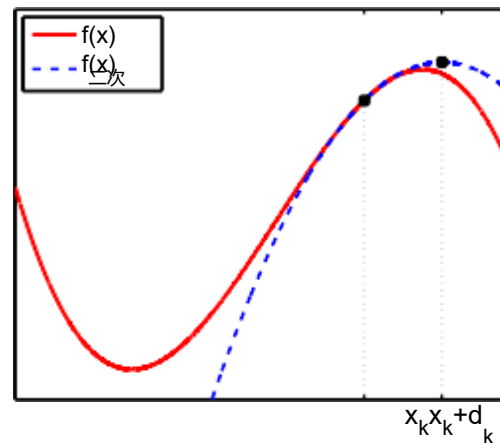
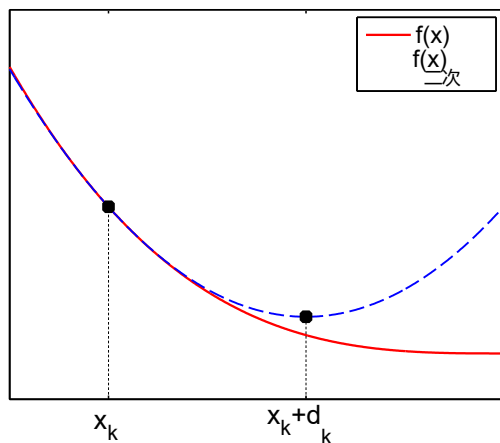
其中, $g_i^{(t)} = \sum_{k=1}^t (\nabla E(\mathbf{w}_i^{(k)}))^2$.

- 在改进版AdaDelta中, 采用固定大小的滑动窗口内梯度平方的指数加权, 而非从1到t所有时刻的梯度

AdaGrad与牛顿法的关联

AdaGrad是对牛顿法优化目标函数的一种近似实现

牛顿法原理：用易于求解的优化问题近似替代真实函数



- 在一种二阶方法如牛顿法中,通过近似目标函数在 $w^{(t)}$ 处用一个二次函数表示,然后最小化这个二次函数

基于二阶泰勒展开的推导

围绕 $\mathbf{w}^{(t)}$ 的二阶泰勒展开

$$\mathcal{E}(\mathbf{w}) \approx \mathcal{E}(\mathbf{w}^{(t)}) + (\mathbf{w} - \mathbf{w}^{(t)})^\top \nabla \mathcal{E}(\mathbf{w}^{(t)}) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^{(t)})^\top \mathbf{H}^{(t)}(\mathbf{w} - \mathbf{w}^{(t)})$$

$$f(x) \approx f(x_t) + f'(x_t)(x - x_t) + \frac{1}{2}f''(x_t)(x - x_t)^2$$

其中

- $\nabla \mathcal{E}(\mathbf{w}^{(t)})$ 是梯度
- $\mathbf{H}^{(t)}$ 是在 $\mathbf{w}^{(t)}$ 处计算的海森矩阵

优化近似函数

最小化二阶泰勒近似

$$\mathcal{E}(\mathbf{w}) \approx \mathcal{E}(\mathbf{w}^{(t)}) + (\mathbf{w} - \mathbf{w}^{(t)})^\top \nabla \mathcal{E}(\mathbf{w}^{(t)}) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^{(t)})^\top \mathbf{H}^{(t)}(\mathbf{w} - \mathbf{w}^{(t)})$$

并将解作为参数的新估计值

$$\mathbf{w}^{(t+1)} \leftarrow \min_{\mathbf{w}} (\mathbf{w} - \mathbf{w}^{(t)})^\top \nabla \mathcal{E}(\mathbf{w}^{(t)}) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^{(t)})^\top \mathbf{H}^{(t)}(\mathbf{w} - \mathbf{w}^{(t)})$$

二次函数的最小化有闭式解,因此得到牛顿法更新规则

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \left(\mathbf{H}^{(t)}\right)^{-1} \nabla \mathcal{E}(\mathbf{w}^{(t)})$$

比SGD更快,但计算海森矩阵逆代价高昂

AdaGrad与牛顿法的关联

- 牛顿法更新规则

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \left(\mathbf{H}^{(t)} \right)^{-1} \nabla \mathcal{E}(\mathbf{w}^{(t)})$$

- AdaGrad通过近似海森矩阵其中 $\mathbf{g}_i^{(t)} = \sum_{k=1}^t (\nabla E(\mathbf{w}_i^{(k)}))^2$ 获取更新规则

$$\mathbf{w}_i^{(t+1)} = \mathbf{w}_i^{(t)} - \frac{\eta}{\sqrt{\mathbf{g}_i^{(t)} + \epsilon}} \nabla E(\mathbf{w}_i^{(t)})$$

Adam

- 自适应矩估计(Adam)是一种方法, 最初由[Kingma,Ba2015]提出——它结合了AdaDelta中使用的自适应学习率思想与一个自适应动量项

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad \text{第一矩梯度估计} \quad (1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \text{第二矩梯度估计(2)}$$

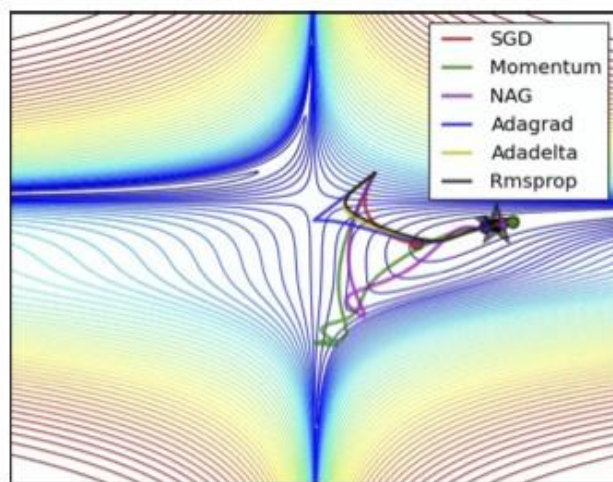
- 模型参数按如下方式更新:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \left(\frac{\eta}{\sqrt{v_t / (1 - \beta_2^t)} + \epsilon} \right) \frac{m_t}{1 - \beta_1^t}$$

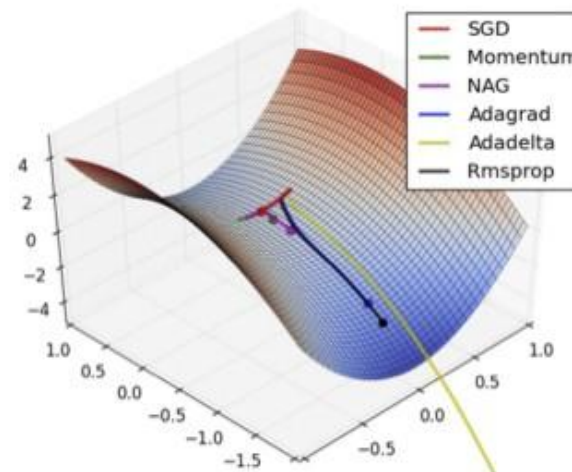
- 其他自适应方法包括RMSprop,AdaMax,Nadam等

应选择哪种优化器

- 对于稀疏参数向量，自适应优化器表现优异，且Adam通常优于其他自适应方法。但具体趋势因数据集和损失函数而异



(a) SGD optimization on loss surface contours



(b) SGD optimization on saddle point

Figure 4: Source and full animations: [Alec Radford](http://www.denizyuret.com/2015/03/alec-radfords-animations-for.html)

完整动画可在此查看：

<http://www.denizyuret.com/2015/03/alec-radfords-animations-for.html>

目录

Contents

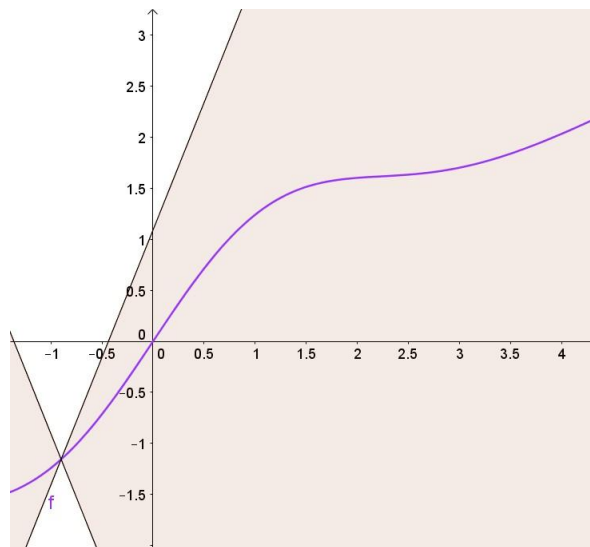
- 1 分布式机器学习概述
- 2 SGD与mini-SGD
- 3 SGD的变体
- 4 目标函数的性质**
- 5 梯度下降的收敛性分析

利普希茨连续性

- 一种更强的连续性形式，限制函数变化速度的上限
- 若函数 $F(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}$ 满足对某个常数 $K > 0$ 有

$$\|F(\mathbf{x}) - F(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\| \text{ 对所有 } \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$$

- 一维函数直观理解：若从任意点 $(x, F(x))$ 引出斜率为 K 和 $-K$ 的直线，整个函数图像需位于这两条直线之间

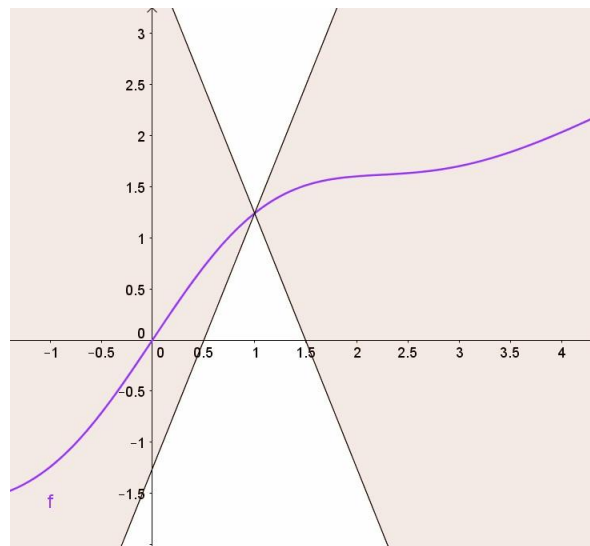


利普希茨连续性

- 一种更强的连续性形式，限制函数变化速度的上限
- 若函数 $F(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}$ 满足对某个常数 $K > 0$ 有

$$\|F(\mathbf{x}) - F(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\| \text{ 对所有 } \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$$

- 一维函数直观理解：若从任意点 $(x, F(x))$ 引出斜率为 K 和 $-K$ 的直线，整个函数图像需位于这两条直线之间

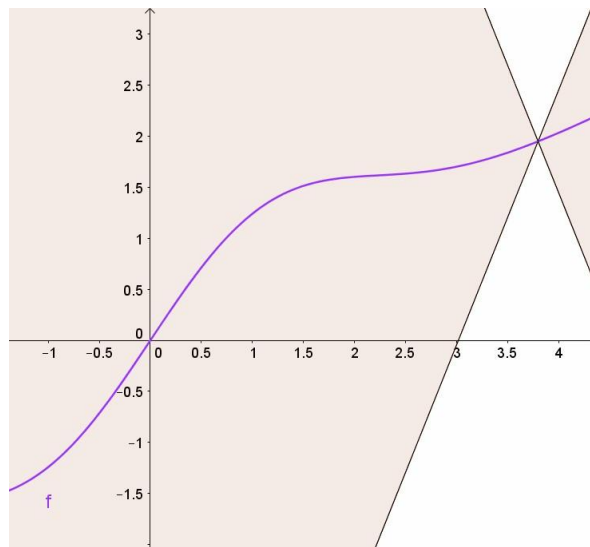


利普希茨连续性

- 一种更强的连续性形式，限制函数变化速度的上限
- 若函数 $F(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}$ 满足对某个常数 $K > 0$ 有

$$\|F(\mathbf{x}) - F(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\| \text{ 对所有 } \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$$

- 一维函数直观理解：若从任意点 $(x, F(x))$ 引出斜率为 K 和 $-K$ 的直线，整个函数图像需位于这两条直线之间



利普希茨光滑性

- 若函数 $F(\mathbf{x})$ 的梯度满足利普希茨连续, 即存在常数 L 使得

$$\|\nabla F(\mathbf{x}) - \nabla F(\mathbf{y})\| \leq L \|\mathbf{x} - \mathbf{y}\|$$

- **直观理解:** 函数斜率变化率受限于 L , 即梯度不会剧烈波动
- **示例:** 下列函数是否利普希茨光滑?

$$F(x) = x^2 \frac{1}{2}$$

$$F'(x) = 2x$$

$$|F'(x) - F'(y)| = 2|x - y|$$

因此, $F(x)$ 是利普希茨光滑函数, $L=2$

利普希茨光滑的等价判定条件

- 若 $F(\mathbf{x})$ 二阶可微且其Hessian矩阵的谱范数有界

$$\|\nabla^2 F(\mathbf{x})\| \leq L \text{ for all } \mathbf{x} \in \mathbb{R}^d$$

则 $F(\mathbf{x})$ 是利普希茨光滑函数，常数为 L

- 矩阵 $\mathbf{H}$ 的谱范数是什么？即 $\mathbf{H}$ 的最大奇异值 $\sigma_{\max}(\mathbf{H}) = \sqrt{\lambda_{\max}(\mathbf{H}^T \mathbf{H})}$
- 示例：以下函数 L 是否光滑？

$$F(\mathbf{x}) = \frac{1}{2} \|\mathbf{A}\mathbf{x} - \mathbf{y}\|^2$$
$$\|\nabla^2 F(\mathbf{x})\|^2 = \|\mathbf{A}\|^2 \leq \sigma_{\max}^2$$

利普希茨光滑的等价判定条件

- 一个函数若满足以下条件，则称其为 L -光滑：对于任意 $\mathbf{x}$ 和 $\mathbf{y}$ ，有上界

$$F(\mathbf{x}) \leq F(\mathbf{y}) + \nabla F(\mathbf{y})^\top (\mathbf{x} - \mathbf{y}) + \frac{L}{2} \|\mathbf{x} - \mathbf{y}\|^2$$

- 这是一个重要的界限，将在SGD收敛性分析中频繁使用
- 更多关于Lipschitz光滑性的条件，请查看：
<http://xingyuzhou.org/blog/notes/Lipschitz-gradient>

强凸性的定义

一个可微函数 $F(\mathbf{x})$ 若满足以下条件，则称其为 c -强凸

$$F(\mathbf{x}) \geq F(\mathbf{y}) + \nabla F(\mathbf{y})^\top (\mathbf{x} - \mathbf{y}) + \frac{1}{2}c\|\mathbf{x} - \mathbf{y}\|^2 \text{ for all } \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$$

强凸性的一个推论：最优性间隙的界限

$$2c(F(\mathbf{x}) - F(\mathbf{x}^*)) \leq \|\nabla F(\mathbf{x})\|^2 \text{ 对所有 } \mathbf{x} \in \mathbb{R}^d$$

这被称为Polyak-Lojasiewicz(PL)不等式。

一个 c -强凸且 L -光滑的函数

满足由以下给出的上下界

$$\begin{aligned} F(\mathbf{x}) &\leq F(\mathbf{y}) + \nabla F(\mathbf{y})^\top (\mathbf{x} - \mathbf{y}) + \frac{L}{2} \|\mathbf{x} - \mathbf{y}\|^2 \\ F(\mathbf{x}) &\geq F(\mathbf{y}) + \nabla F(\mathbf{y})^\top (\mathbf{x} - \mathbf{y}) + \frac{1}{2} c \|\mathbf{x} - \mathbf{y}\|^2 \text{ for all } \mathbf{x}, \mathbf{y} \in \mathbb{R}^d \end{aligned}$$

注意到 $L \geq c$ 必须满足才能保证上述两个不等式同时成立的条件

目录

Contents

- 1 分布式机器学习概述
- 2 SGD与mini-SGD
- 3 SGD的变体
- 4 目标函数的性质
- 5 梯度下降的收敛性分析**

收敛性保证的比较

算法	假设条件	收敛速率
批量梯度下降	L -光滑且 c -强凸	$O(\log(1/\epsilon))$
小批量随机梯度下降	L -光滑且 c -强凸	$O(1/\epsilon)$
小批量随机梯度下降	L -光滑, 非凸	$O(1/\epsilon^2)$

随机平均梯度法(SAG)

核心思想：利用随机梯度仍能获得 $O(\log(1/\epsilon))$ 的收敛速率

- 批量梯度下降法

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{1}{N} \sum_{n=1}^N \nabla f(\mathbf{x}_t; \xi_n)$$

其中每次迭代需计算全部 N 个梯度

$\nabla f(\mathbf{x}_t, \xi_n)$ ($n=1, \dots, N$)

- 随机平均梯度(SAG)在每次迭代中，对于一个样本索引被均匀随机选取，

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \left(\frac{\nabla f(\mathbf{x}_t; \xi_i) - v_i}{N} + \frac{1}{N} \sum_{n=1}^N v_n \right)$$
$$v_i = \nabla f(\mathbf{x}_t; \xi_i)$$

其中 $v=[v_1, \dots, v_N]$ 是一组存储在内存中的（可能已过时的）梯度向量集合。

随机平均梯度法(SAG)

核心思想：利用随机梯度仍能获得 $O(\log(1/\epsilon))$ 的收敛速率

- 随机平均梯度(SAG)在每次迭代中，对于一个样本索引被均匀随机选取，

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \left(\frac{\nabla f(\mathbf{x}_t; \xi_i) - v_i}{N} + \frac{1}{N} \sum_{n=1}^N v_n \right)$$
$$v_i = \nabla f(\mathbf{x}_t; \xi_i)$$

其中 $v=[v_1, \dots, v_N]$ 是一组存储在内存中的（可能已过时的）梯度向量集合。

- 优点：只需在每次迭代中计算一个梯度，但仍能获得 $O(\log(1/\epsilon))$ 的收敛速率
- 缺点：需要在内存中维护 N 个梯度 $v=[v_1, \dots, v_N]$ ，这对于较大的 N 可能代价高昂

SAGA: 无偏版本的SAG

- SAG的一个缺点是其梯度存在偏差, 因为 $\frac{1}{N} \sum_{n=1}^N v_n$ 是对 $\frac{1}{N} \sum_{n=1}^N \nabla f(\mathbf{x}_t)$ 的有偏估计
- SAGA是一种改进的无偏更新规则。在每次迭代中, 对于一个样本索引被均匀随机选取,

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \left(\nabla f(\mathbf{x}_t; \xi_i) - v_i + \frac{1}{N} \sum_{n=1}^N v_n \right)$$
$$v_i = \nabla f(\mathbf{x}_t; \xi_i)$$

- 赋予新梯度更高权重。此更新是无偏的, 但相比SAG梯度具有更高方差
- **优点:** 只需在每次迭代中计算一个梯度, 但仍能获得 $O(\log(1/\epsilon))$ 的收敛速率
- **缺点:** 需要在内存中维护 N 个梯度 $\mathbf{v}=[v_1, \dots, v_N]$, 这对于较大的 N 可能代价高昂

随机方差缩减梯度(SVRG)

核心思想：定期计算全量梯度并利用其降低随机梯度的方差

- 批量梯度下降法

$$\begin{aligned}\mathbf{x}_{t+1} &= \mathbf{x}_t - \eta \nabla F(\mathbf{x}_t) \\ &= \mathbf{x}_t - \eta \frac{1}{N} \sum_{n=1}^N \nabla f(\mathbf{x}_t, \xi_n)\end{aligned}$$

其中在每次迭代中需要计算N个梯度 $\nabla f(\mathbf{x}_t, \xi_n)$

- 随机方差缩减梯度(SVRG)

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta (g(\mathbf{x}_t, \xi_t) - g(\tilde{\mathbf{x}}, \xi_t) + \nabla F(\tilde{\mathbf{x}}))$$

其中 $\tilde{\mathbf{x}}$ 是 $\mathbf{x}$ 的快照，每 t_0 次迭代后更新一次。此时还需计算全梯度 $\nabla F(\tilde{\mathbf{x}})$.

随机方差缩减梯度(SVRG)

核心思想： 定期计算全量梯度并利用其降低随机梯度的方差

- 随机方差缩减梯度(SVRG)

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta (g(\mathbf{x}_t, \xi_t) - g(\tilde{\mathbf{x}}, \xi_t) + \nabla F(\tilde{\mathbf{x}}))$$

其中 $\tilde{\mathbf{x}}$ 是 $\mathbf{x}$ 的快照，每 t_0 次迭代后更新一次。此时还需计算全梯度 $\nabla F(\tilde{\mathbf{x}})$ 。

- **优点：** 无需在内存中存储 N 个梯度，仍能获得 $O(\log(1/\epsilon))$ 的收敛速率
- **缺点：** 需定期计算全梯度 $\nabla F(\tilde{\mathbf{x}})$ ，当 N 较大时计算成本较高

SGD变体对比

算法	收敛速率	每次迭代计算量	内存占用
批量梯度下降	$O(\log(1/\epsilon))$	$O(Nd)$	$O(d)$
SGD	$O(1/\epsilon)$	$O(d)$	$O(d)$
小批量随机梯度下降	$O(1/\epsilon)$	$O(bd)$	$O(d)$
SAG	$O(\log(1/\epsilon))$	$O(d)$	$O(Nd)$
SAGA	$O(\log(1/\epsilon))$	$O(d)$	$O(Nd)$
SVRG	$O(\log(1/\epsilon))$	$O(Nd/t_0)$	$O(d)$